

Numpy Notes

Introduction:

- NumPy is a Python library.
- NumPy is used for working with arrays.
- NumPy is short for "Numerical Python"
- Support for large, multi-dimensional arrays and matrices

Why Use NumPy?

- **Faster Computation:** NumPy is much faster than Python lists due to its efficient memory management and vectorized operations.
- **Convenience:** It provides built-in functions for mathematical operations, statistics, and linear algebra.
- **Integration:** Works well with libraries like Pandas, Matplotlib, TensorFlow, and SciPy.

Installing NumPy:

- To install NumPy, use:
 1. **pip install numpy**
- Verify installation in Python:
 1. `import numpy as np`
 2. `print(np.__version__)`

Creating Arrays

- The array object in NumPy is called **ndarray**. Also can create arrays from lists:
 1. `import numpy as np`
 2. **#0-D Arrays or Scalars**
 3. `arr = np.array(42)`
 4. `print(arr)`
 5. **# 1D array or uni-dimensional**
 6. `arr1 = np.array([1, 2, 3, 4, 5])`
 7. `print(arr1)`
 8. **# 2D array (Matrix)**
 9. `arr2 = np.array([[1, 2, 3], [4, 5, 6]])`
 10. `print(arr2)`
 11. `print(type(arr))`
 12. **#3-D arrays**

```
13. arr = np.array([[[1, 2, 3], [4, 5, 6]], [[1, 2, 3], [4, 5, 6]]])
14. print(arr)
15. arr = np.array([1, 2, 3, 4], ndmin=5) # can define the number of dimensions by using
    the ndmin argument
16. print(arr)
```

Checking Array Properties:

- `print(arr1.ndim)` # Number of dimensions
 - `print(arr2.shape)` # Shape (rows, columns)
 - `print(arr2.size)` # Total number of elements
 - `print(arr2.dtype)` # Data type of elements
-

Creating Special Arrays:

- `np.zeros((3, 3))` # 3x3 matrix of zeros
- `np.ones((2, 4))` # 2x4 matrix of ones
- `np.full((2, 3), 7)` # 2x3 matrix filled with 7
- `np.eye(4)` # Identity matrix (4x4)
- **Using `arange()` and `linspace()`:**
- Both `np.arange()` and `np.linspace()` are used to create sequences of numbers efficiently.

`np.arange(start, stop, step):`

- This function generates evenly spaced values within a given range, similar to Python's `range()` but for NumPy arrays.

Usage:

- Used when you know the **step size** between values.
- Stops **before** the given stop value.
- the step size is **fixed**
- don't need an exact number of elements.

Example:

```
1. import numpy as np
2. arr = np.arange(1, 10, 2)
3. print(arr) # Output: [1, 3, 5, 7, 9]
```

Explanation:

- Starts at 1, ends **before** 10, and increases by 2.
- Works well when you need a sequence with a fixed step.

`np.linspace(start, stop, num_points):`

- This function generates `num_points` evenly spaced values between start and stop, **including the stop value**.

Usage:

- Used when you know **how many points** you want, rather than the step size.
- Always includes the stop value.

Example:

1. `arr = np.linspace(0, 1, 5)`
2. `print(arr)` # Output: `[0. 0.25 0.5 0.75 1. ]`

Explanation:

- Starts at 0, ends at 1, and generates 5 equally spaced values.
- Unlike `arange()`, it **includes** 1.

When to Use `linspace()`?

- When you need a **specific number** of points, not a fixed step size.
 - Useful in plotting graphs and scientific computations (e.g., time intervals in physics).
-

Reshaping and Flattening:

Reshape Arrays

- `arr = np.arange(1, 10)`
- `reshaped_arr = arr.reshape(3, 3)`
- `print(reshaped_arr)`
- # Convert the following 1-D array with 12 elements into a 3-D array
- `arr = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12])`
- `newarr = arr.reshape(2, 3, 2)`
- `print(newarr)`

Flattening Arrays:

Flattening array means converting a multidimensional array into a 1D array.
use `reshape(-1)` to do this.

1. `import numpy as np`
 2. `arr = np.array([[1, 2, 3], [4, 5, 6]])`
 3. `newarr = arr.reshape(-1)`
 4. `print(newarr)`
 5. `flattened = reshaped_arr.flatten()`
 6. `print(flattened)` # Converts to 1D
-

Unknown Dimension in reshape()

- In NumPy, you can use -1 in the reshape() method to let NumPy **automatically calculate** the missing dimension.

Example:

1. import numpy as np
 2. arr = np.array([1, 2, 3, 4, 5, 6, 7, 8])
 3. newarr = arr.reshape(2, 2, -1)
 4. print(newarr)
-

Indexing and Slicing:

Indexing (Accessing Elements)

1. arr = np.array([10, 20, 30, 40, 50])
2. print(arr[0]) # First element
3. print(arr[-1]) # Last element

Slicing (Extracting Parts of an Array)

1. print(arr[1:4]) # Elements from index 1 to 3
2. print(arr[:3]) # First 3 elements
3. print(arr[::2]) # Every second element

Modifying Elements

1. arr[2] = 99
 2. print(arr) # [10 20 99 40 50]
-

Basic Operations on Arrays

Element-wise Arithmetic Operations

- a = np.array([10, 20, 30])
- b = np.array([1, 2, 3])
- print(a + b) # [11 22 33]
- print(a - b) # [9 18 27]
- print(a * b) # [10 40 90]
- print(a / b) # [10. 10. 10.]
- print(a ** 2) # [100 400 900]

Note: The arrays must have the **same shape** (or be broadcastable).

Mathematical Functions

- np.sin(a), np.cos(a), np.exp(a), np.sqrt(a)

Indexing and Slicing

Indexing (Accessing Elements)

```
arr = np.array([10, 20, 30, 40, 50])
print(arr[0]) # First element
print(arr[-1]) # Last element
```

Accessing Elements in 1D Arrays

- Indexing starts at **0**.
- Example:
 1. import numpy as np
 2. arr = np.array([1, 2, 3, 4])
 3. print(arr[0]) # First element (1)
 4. print(arr[1]) # Second element (2)
 5. print(arr[2] + arr[3]) # Third + Fourth element (3 + 4 = 7)

Accessing Elements in 2D Arrays (Rows & Columns)

- Use `arr[row, column]` format.
- Example:
 1. arr = np.array([[1, 2, 3, 4, 5], [6, 7, 8, 9, 10]])
 2. print(arr[0, 1]) # 2nd element in 1st row (2)
 3. print(arr[1, 4]) # 5th element in 2nd row (10)

Accessing Elements in 3D Arrays

- Use `arr[depth, row, column]`.
- Example:
 1. arr = np.array([[[1, 2, 3], [4, 5, 6]], [[7, 8, 9], [10, 11, 12]]])
 2. print(arr[0, 1, 2]) # 6 (from first 3D layer, second row, third column)

Negative Indexing (Access from the end)

- -1 refers to the last element.
- Example:
 - print(arr[1, -1]) # Last element of the 2nd row (10)

Slicing (Extracting Parts of an Array)

- print(arr[1:4]) # Elements from index 1 to 3
- print(arr[:3]) # First 3 elements

- `print(arr[::2])` # Every second element

Slicing means selecting a portion of an array. The syntax is:

- `arr[start:end:step]`
 - **start** → Starting index (default is 0)
 - **end** → Ending index (excluded)
 - **step** → Step size (default is 1)
-

1D Array Slicing

Example 1: Get elements from index **1 to 5** (excluding 5)

1. `import numpy as np`
2. `arr = np.array([1, 2, 3, 4, 5, 6, 7])`
3. `print(arr[1:5])` # [2, 3, 4, 5]

Example 2: Get elements from index **4 to the end**

- `print(arr[4:])` # [5, 6, 7]

Example 3: Get elements from the **start to index 4** (excluding 4)

- `print(arr[:4])` # [1, 2, 3, 4]
-

Negative Slicing (Count from the end)

Example 4: Get elements from **3rd last to 2nd last**

- `print(arr[-3:-1])` # [5, 6]
-

Using Step in Slicing

Example 5: Get every **2nd element** from index **1 to 5**

- `print(arr[1:5:2])` # [2, 4]

Example 6: Get every **2nd element** from the entire array

- `print(arr[::2])` # [1, 3, 5, 7]
-

Slicing 2D Arrays

Example 7: Slice 2nd row (index 1), elements from index 1 to 4

1. `arr2 = np.array([[1, 2, 3, 4, 5], [6, 7, 8, 9, 10]])`
2. `print(arr2[1, 1:4])` # [7, 8, 9]

Example 8: Get index 2 from both rows

- `print(arr2[0:2, 2])` # [3, 8]

Example 9: Slice index 1 to 4 from both rows

1. `print(arr2[0:2, 1:4])`
2. # Output:
3. # [[2, 3, 4]
4. # [7, 8, 9]]

Modifying Elements

1. `arr[2] = 99`
 2. `print(arr)` # [10 20 99 40 50]
-

Broadcasting

NumPy allows operations between arrays of different shapes.

1. `a = np.array([[1, 2, 3], [4, 5, 6]])`
2. `b = np.array([10, 20, 30])`
3. `print(a + b)`

Here, b is **broadcasted** to match a's shape.

Aggregation Functions

1. `arr = np.array([[1, 2, 3], [4, 5, 6]])`
2. `print(np.sum(arr))` # Sum of all elements
3. `print(np.mean(arr))` # Mean (average)
4. `print(np.min(arr))` # Minimum value
5. `print(np.max(arr))` # Maximum value
6. `print(np.std(arr))` # Standard deviation
7. `print(np.sum(arr, axis=0))` # Sum along columns
8. `print(np.sum(arr, axis=1))` # Sum along rows

Working with Random Numbers

- NumPy provides a random module for generating random numbers.

Generating Random Numbers

1. `np.random.rand(3, 3)` # 3x3 matrix of random numbers between 0 and 1
2. `np.random.randint(1, 10, (3, 3))` # 3x3 matrix of random integers

Shuffling and Permutations

1. `arr = np.array([1, 2, 3, 4, 5])`
2. `np.random.shuffle(arr)` # Shuffle the array in place
3. `print(arr)`

Linear Algebra with NumPy

- NumPy supports various matrix operations.

Matrix Multiplication

1. `A = np.array([[1, 2], [3, 4]])`
2. `B = np.array([[5, 6], [7, 8]])`
3. `print(np.dot(A, B))` # Matrix multiplication
4. `print(A @ B)` # Alternative method

Determinant and Inverse

1. `np.linalg.det(A)` # Determinant
2. `np.linalg.inv(A)` # Inverse

NumPy vs Lists (Performance Test)

1. `import time`
2. `size = 1000000`
3. `list1 = list(range(size))`
4. `list2 = list(range(size))`
5. `array1 = np.array(list1)`
6. `array2 = np.array(list2)`
7. # Python List
8. `start = time.time()`
9. `result = [x + y for x, y in zip(list1, list2)]`
10. `end = time.time()`
11. `print("List Time:", end - start)`

```
12. # NumPy Array
13. start = time.time()
14. result = array1 + array2
15. end = time.time()
16. print("NumPy Time:", end - start)
```

NumPy Data Types

NumPy introduces additional data types with **single-character codes**:

Character	Data Type
I	Integer
U	Unsigned Integer
F	Float
C	Complex Float
B	Boolean
M	Timedelta
M	Datetime
O	Object
S	String (bytes)
U	Unicode String
V	Raw memory (void)

Checking the Data Type of a NumPy Array

- Use `.dtype` to check the type of a NumPy array.

Example 1: Checking the type of an integer array

```
1. import numpy as np
2. arr = np.array([1, 2, 3, 4])
3. print(arr.dtype) # int32 or int64 (depends on system)
```

Example 2: Checking the type of a string array

```
1. arr = np.array(['apple', 'banana', 'cherry'])
2. print(arr.dtype) # <U6 (Unicode string of max length 6)
```

Creating Arrays with a Specific Data Type

- You can define the data type using the `dtype` parameter.

Example 3: Create an array with string type (s for byte string)

```
1. arr = np.array([1, 2, 3, 4], dtype='S')
```

2. `print(arr)` `# [b'1' b'2' b'3' b'4']`
3. `print(arr.dtype)` `# dtype('S1')`

Example 4: Create an integer array of **4-byte integers (i4)**

1. `arr = np.array([1, 2, 3, 4], dtype='i4')`
 2. `print(arr)`
 3. `print(arr.dtype)` `# int32`
-

Handling Type Errors in NumPy

- If elements **can't be converted**, NumPy raises a **ValueError**.

Example 5: Trying to convert a string to an integer (this fails)

1. `arr = np.array(['a', '2', '3'], dtype='i')`
2. `# ValueError: invalid literal for int() with base 10: 'a'`

NumPy Array Iteration

What is Iteration?

- Iteration means going through each element of an array one by one.

1. Iterating 1D Arrays

- A simple `for` loop can be used to iterate through a 1D NumPy array.
1. `import numpy as np`
 2. `arr = np.array([1, 2, 3])`
 3. `for x in arr:`
 4. `print(x)`

2. Iterating 2D Arrays

- In a 2D array, iterating goes through each row first.

1. `arr = np.array([[1, 2, 3], [4, 5, 6]])`
2. `for x in arr:`
3. `print(x)` `# Prints each row as an array`

To access each scalar (individual element), use nested loops:

1. `for x in arr:`
2. `for y in x:`
3. `print(y)`

3. Iterating 3D Arrays

- For a 3D array, iteration goes through each 2D sub-array.
1. `arr = np.array([[[1, 2, 3], [4, 5, 6]], [[7, 8, 9], [10, 11, 12]]])`
 2. `for x in arr:`
 3. `print(x)` # Prints each 2D sub-array
- To get each scalar value:
 - `for x in arr:`
 - `for y in x:`
 - `for z in y:`
 - `print(z)`

4. Iterating with `nditer()` (Simplified Iteration)

- Instead of using multiple loops, `nditer()` makes it easier.
- `for x in np.nditer(arr):`
- `print(x)`

5. Iterating with Different Data Types

- Convert elements to strings while iterating using `op_dtypes` and `flags=['buffered']`.
1. `arr = np.array([1, 2, 3])`
 2. `for x in np.nditer(arr, flags=['buffered'], op_dtypes=['S']):`
 3. `print(x)`

6. Iterating with a Step Size

- Skip elements while iterating using slicing.
1. `arr = np.array([[1, 2, 3, 4], [5, 6, 7, 8]])`
 2. `for x in np.nditer(arr[:, ::2]):` # Skips every other element
 3. `print(x)`

7. Enumerating Elements using `ndenumerate()`

- This method provides both **index** and **value** while iterating.

1D Array Example:

1. `arr = np.array([1, 2, 3])`
2. `for idx, x in np.ndenumerate(arr):`
3. `print(idx, x)` # Prints (index, value)

2D Array Example:

1. `arr = np.array([[1, 2, 3, 4], [5, 6, 7, 8]])`
2. `for idx, x in np.ndenumerate(arr):`
3. `print(idx, x)` # Prints index and value for each element

Conclusion

- Use basic loops for small arrays.
 - Use `nditer()` for simpler iteration.
 - Use `ndenumerate()` to get **index** while iterating.
 - Use **slicing** (`::step_size`) to skip elements.
-

NumPy Array: Copy vs. View

The Difference Between Copy and View:

- The main difference between a copy and a view of an array is that the copy is a new array, and the view is just a view of the original array.
- The copy *owns* the data and any changes made to the copy will not affect original array, and any changes made to the original array will not affect the copy.
- The view *does not own* the data and any changes made to the view will affect the original array, and any changes made to the original array will affect the view.

Copy Example: Changes Do Not Affect the Original

1. `import numpy as np`
 2. `arr = np.array([1, 2, 3, 4, 5])`
 3. `x = arr.copy()` # Creates a new independent copy
 4. `arr[0] = 42` # Changing original array
 5. `print(arr)` # [42, 2, 3, 4, 5]
 6. `print(x)` # [1, 2, 3, 4, 5] (Unchanged)
- **Copy is independent** → Changing `arr` **does not** affect `x`.

View Example: Changes Affect Both

1. `import numpy as np`
2. `arr = np.array([1, 2, 3, 4, 5])`
3. `x = arr.view()` # Creates a view (reference to original data)
4. `arr[0] = 42` # Changing original array
5. `print(arr)` # [42, 2, 3, 4, 5]
6. `print(x)` # [42, 2, 3, 4, 5] (Changed!)

- **View is linked** → Changing `arr` **also changes** `x`.

Modifying the View: Affects the Original

```
1. import numpy as np
2. arr = np.array([1, 2, 3, 4, 5])
3. x = arr.view() # Create a view
4. x[0] = 31 # Changing view
5. print(arr) # [31, 2, 3, 4, 5]
6. print(x) # [31, 2, 3, 4, 5]
```

- **View updates the original** → Changes in `x` **reflect in** `arr`.

Check if an Array Owns Its Data (`base` Attribute)

- Every NumPy array has the attribute `base`
- If **`base`** is **`None`** → It's a copy (owns data).
- If **`base`** points to another array → It's a view (doesn't own data).

```
1. import numpy as np
2. arr = np.array([1, 2, 3, 4, 5])
3. x = arr.copy()
4. y = arr.view()
5. print(x.base) # None (Copy owns its data)
6. print(y.base) # [1, 2, 3, 4, 5] (View points to arr)
```

Joining Arrays in NumPy

- In **NumPy**, "joining" means combining two or more arrays into a single array. There are different ways to join arrays depending on how you want to merge them.

Using `concatenate()`

- This method joins two or more arrays along an existing axis.
- `axis=0` → Joins along rows (vertically)
- `axis=1` → Joins along columns (horizontally)

Example 1: Joining 1-D arrays

```
1. import numpy as np
2. arr1 = np.array([1, 2, 3])
3. arr2 = np.array([4, 5, 6])
4. arr = np.concatenate((arr1, arr2))
5. print(arr)
```

Output:

- [1 2 3 4 5 6]
 - The two arrays are **merged into a single 1-D array**.
-

Example 2: Joining 2-D arrays along columns (`axis=1`)

1. `import numpy as np`
2. `arr1 = np.array([[1, 2], [3, 4]])`
3. `arr2 = np.array([[5, 6], [7, 8]])`
4. `arr = np.concatenate((arr1, arr2), axis=1)`
5. `print(arr)`

Output:

- `[[1 2 5 6]`
 - `[3 4 7 8]]`
 - The arrays are **joined side by side** (column-wise).
-

Using `stack()`

- Stacking is similar to concatenation, but it **creates a new axis** while joining.

Example: Stacking along a new axis

1. `import numpy as np`
2. `arr1 = np.array([1, 2, 3])`
3. `arr2 = np.array([4, 5, 6])`
4. `arr = np.stack((arr1, arr2), axis=1)`
5. `print(arr)`

Output:

1. `[[1 4]`
 2. `[2 5]`
 3. `[3 6]]`
- The arrays are stacked **column-wise**, creating a 2-D structure.
-

Using Special Stacking Functions

- NumPy provides **shortcuts** for stacking arrays in different directions.

Stacking Horizontally (`hstack()`)

- Joins arrays **side by side** (like `axis=1` in `concatenate()`).
1. `import numpy as np`
 2. `arr1 = np.array([1, 2, 3])`
 3. `arr2 = np.array([4, 5, 6])`
 4. `arr = np.hstack((arr1, arr2))`
 5. `print(arr)`

Output:

```
[1 2 3 4 5 6]
```

- Works the same as `concatenate()` for 1-D arrays.
-

Stacking Vertically (`vstack()`)

- Joins arrays **one on top of another** (like `axis=0` in `concatenate()`).
1. `import numpy as np`
 2. `arr1 = np.array([1, 2, 3])`
 3. `arr2 = np.array([4, 5, 6])`
 4. `arr = np.vstack((arr1, arr2))`
 5. `print(arr)`

Output:

```
[[1 2 3]
 [4 5 6]]
```

- The two 1-D arrays become rows of a 2-D array.
-

Stacking Along Depth (`dstack()`)

- Joins arrays **along the third dimension (depth)**.
1. `import numpy as np`
 2. `arr1 = np.array([1, 2, 3])`
 3. `arr2 = np.array([4, 5, 6])`
 4. `arr = np.dstack((arr1, arr2))`
 5. `print(arr)`

Output:

- ```
[[[1 4]
```

- [2 5]
- [3 6]]]
- The arrays are **stacked in a 3D format**.

---

## Summary of Functions

| Method        | How it Joins                                           |
|---------------|--------------------------------------------------------|
| concatenate() | Joins existing arrays along a given axis               |
| stack()       | Creates a new axis while joining                       |
| hstack()      | Joins arrays <b>horizontally (side by side)</b>        |
| vstack()      | Joins arrays <b>vertically (one on top of another)</b> |
| dstack()      | Joins arrays <b>along depth (3D stacking)</b>          |

## Splitting NumPy Arrays

- Splitting is the **opposite of joining**. It breaks one large array into multiple smaller ones.

### 1. Using `array_split()`

- We specify how many parts we want.

#### *Example: Splitting a 1D Array*

1. `arr = np.array([1, 2, 3, 4, 5, 6])`
2. `newarr = np.array_split(arr, 3)` # Splits into 3 parts
3. `print(newarr)` # Output: [array([1, 2]), array([3, 4]), array([5, 6])]

If the number of elements doesn't divide evenly, NumPy adjusts the split.

#### *Example: Splitting a 2D Array*

1. `arr = np.array([[1, 2], [3, 4], [5, 6], [7, 8]])`
2. `newarr = np.array_split(arr, 2)` # Splits into 2 parts
3. `print(newarr)`

---

## 2. Using Specialized Splitting Functions

NumPy also provides functions similar to `hstack()` and `vstack()`:

- `hsplit()` → Splits along columns
- `vsplit()` → Splits along rows
- `dsplit()` → Splits along depth

*Example: Using `hsplit()`*

1. `arr = np.array([[1, 2, 3], [4, 5, 6]])`
2. `newarr = np.hsplit(arr, 3) # Splits into 3 column arrays`
3. `print(newarr)`

---

## Summary

| Operation    | Function                                                                                                                  |
|--------------|---------------------------------------------------------------------------------------------------------------------------|
| Join Arrays  | <code>concatenate()</code> , <code>stack()</code> , <code>hstack()</code> , <code>vstack()</code> , <code>dstack()</code> |
| Split Arrays | <code>array_split()</code> , <code>hsplit()</code> , <code>vsplit()</code> , <code>dsplit()</code>                        |

## 1. Changing Array Shape

These functions help convert multi-dimensional arrays into 1D arrays.

`flatten()`

- Returns a **copy** of the array as a 1D array.
- Changes do **not** affect the original array.

1. `import numpy as np`
2. `arr = np.array([[1, 2, 3], [4, 5, 6]])`
3. `flat_arr = arr.flatten()`
4. `print(flat_arr) # Output: [1 2 3 4 5 6]`

`ravel()`

- Returns a **view** of the original array (if possible).
- Changes affect the original array.

1. `ravel_arr = arr.ravel()`
2. `ravel_arr[0] = 99`
3. `print(arr)`

---

## 2. Rearranging Elements

NumPy offers functions to **rotate and flip** arrays.

`rot90()` - Rotate an array by 90 degrees

1. `arr = np.array([[1, 2], [3, 4]])`

2. `rotated = np.rot90(arr)`
3. `print(rotated)`

#### `flip()` - Flip the array along a given axis

1. `arr = np.array([[1, 2], [3, 4]])`
2. `flipped = np.flip(arr) # Flips all elements`
3. `print(flipped)`

#### `fliplr()` - Flip left to right (horizontal)

1. `flipped_lr = np.fliplr(arr)`
2. `print(flipped_lr)`

#### `flipud()` - Flip up to down (vertical)

1. `flipped_ud = np.flipud(arr)`
2. `print(flipped_ud)`

---

## Summary Table

| Function                      | Description                                             |
|-------------------------------|---------------------------------------------------------|
| <b><code>flatten()</code></b> | Returns a <b>copy</b> of a 1D array                     |
| <b><code>ravel()</code></b>   | Returns a <b>view</b> of a 1D array (modifies original) |
| <b><code>rot90()</code></b>   | Rotates the array by <b>90 degrees</b>                  |
| <b><code>flip()</code></b>    | Flips elements along <b>all axes</b>                    |
| <b><code>fliplr()</code></b>  | Flips elements <b>left-to-right</b>                     |
| <b><code>flipud()</code></b>  | Flips elements <b>up-down</b>                           |

## Searching in NumPy Arrays

- In NumPy, you can **search for values** in an array and **find their positions (indexes)**. There are two main ways to do this:
    1. **Using `where()`** → To find existing values.
    2. **Using `searchsorted()`** → To find where a value should be inserted in a sorted array.
-

## 1. Finding Values in an Array (`where()`)

- The `np.where()` function **returns the indexes** where a certain condition is **True**.

### Find all positions of a specific value

Example: Find the indexes where the value is 4 in the array.

```
1. import numpy as np
2. arr = np.array([1, 2, 3, 4, 5, 4, 4])
3. x = np.where(arr == 4)
4. print(x) # Output: (array([3, 5, 6]),)
```

👉 This means the number **4** appears at **indexes 3, 5, and 6**.

---

### Find indexes of even numbers

```
1. arr = np.array([1, 2, 3, 4, 5, 6, 7, 8])
2. x = np.where(arr % 2 == 0)
3. print(x) # Output: (array([1, 3, 5, 7]),)
```

👉 This shows that even numbers are at **indexes 1, 3, 5, and 7**.

---

### Find indexes of odd numbers

```
1. arr = np.array([1, 2, 3, 4, 5, 6, 7, 8])
2. x = np.where(arr % 2 == 1)
3. print(x) # Output: (array([0, 2, 4, 6]),)
```

👉 This means odd numbers are found at **indexes 0, 2, 4, and 6**.

---

## 2. Finding Where a Value Should Be Inserted (`searchsorted()`)

This function helps find the **correct position** to insert a value **while keeping the array sorted**.

### Find where 7 should be inserted

```
1. arr = np.array([6, 7, 8, 9])
2. x = np.searchsorted(arr, 7)
3. print(x) # Output: 1
```

👉 The number 7 should be inserted at **index 1** to keep the order.

---

## Searching from the Right Side

- By default, `searchsorted()` starts from the **leftmost position**.
- If you want to insert the value at the **rightmost position**, use `side='right'`.

```
1. arr = np.array([6, 7, 8, 9])
2. x = np.searchsorted(arr, 7, side='right')
3. print(x) # Output: 2
```

👉 The number 7 should be inserted at **index 2** (instead of 1).

---

## Finding insertion positions for multiple values

```
1. You can search for multiple values at once using an array.
2. arr = np.array([1, 3, 5, 7])
3. x = np.searchsorted(arr, [2, 4, 6])
4. print(x) # Output: [1 2 3]
```

👉 This means:

- 2 should go at **index 1**
  - 4 should go at **index 2**
  - 6 should go at **index 3**
- 

## 💡 Summary

| Function                                           | Purpose                                   | Example Output                      |
|----------------------------------------------------|-------------------------------------------|-------------------------------------|
| <code>np.where(arr == x)</code>                    | Find all indexes of x in arr              | <code>(array([3, 5, 6]),)</code>    |
| <code>np.where(arr % 2 == 0)</code>                | Find indexes of even numbers              | <code>(array([1, 3, 5, 7]),)</code> |
| <code>np.searchsorted(arr, x)</code>               | Find the index where x should be inserted | 1                                   |
| <code>np.searchsorted(arr, x, side='right')</code> | Find the rightmost insertion index        | 2                                   |

---

---

```
np.searchsorted(arr, [x, y, z])
```

Find insertion indexes for multiple values [1 2 3]

---

## Sorting in NumPy

- Sorting means arranging elements in **ascending** or **descending order**.
  - In NumPy, you can use the `sort()` function to **sort arrays**.
- 

### 1. Sorting a 1D Array

- The `np.sort()` function **returns a sorted copy** of the array (it does not change the original array).

#### Sorting numbers

1. `import numpy as np`
2. `arr = np.array([3, 2, 0, 1])`
3. `print(np.sort(arr))`

👉 The numbers are sorted in **ascending order**.

---

#### Sorting strings alphabetically

1. `arr = np.array(['banana', 'cherry', 'apple'])`
2. `print(np.sort(arr))`

👉 The words are sorted in **alphabetical order**.

---

#### Sorting boolean values

1. `arr = np.array([True, False, True])`
2. `print(np.sort(arr))`

👉 False is treated as 0, and True as 1, so they are sorted as **[False, True, True]**.

---

### 2. Sorting a 2D Array

- When sorting a **2D array**, NumPy **sorts each row separately**.

1. `arr = np.array([[3, 2, 4], [5, 0, 1]])`
2. `print(np.sort(arr))`

👉 Each row is sorted **individually**.

---

## 💡 Summary

| Function                              | Purpose                          | Example Output                           |
|---------------------------------------|----------------------------------|------------------------------------------|
| <code>np.sort(arr)</code>             | Sorts numbers in ascending order | <code>[0 1 2 3]</code>                   |
| <code>np.sort(arr) (Strings)</code>   | Sorts words alphabetically       | <code>['apple' 'banana' 'cherry']</code> |
| <code>np.sort(arr) (Booleans)</code>  | Sorts False before True          | <code>[False True True]</code>           |
| <code>np.sort(arr) (2D Arrays)</code> | Sorts each row separately        | <code>[[2 3 4], [0 1 5]]</code>          |

---

## Filtering in NumPy (Easy Explanation)

- Filtering means **selecting specific elements** from an array based on a condition.
- In **NumPy**, filtering is done using **Boolean indexing**, where:
  1. True means **include** the element in the new array.
  2. False means **exclude** the element.

---

### 1. Filtering with a Boolean List

- You can **manually** define which elements to keep.
  1. `import numpy as np`
  2. `arr = np.array([41, 42, 43, 44])`
  3. `filter_arr = [True, False, True, False] # Keep 41 and 43`
  4. `newarr = arr[filter_arr]`
  5. `print(newarr)`

👉 Only the values **at index 0 and 2** are kept because their corresponding values in `filter_arr` are True.

---

## 2. Creating a Filter Dynamically

- Instead of manually writing True and False, we can generate them based on conditions.

### Keep values greater than 42

```
1. arr = np.array([41, 42, 43, 44])
2. # Generate filter list based on condition
3. filter_arr = []
4. for element in arr:
5. if element > 42:
6. a. filter_arr.append(True)
7. else:
8. a. filter_arr.append(False)
9. newarr = arr[filter_arr]
10. print(newarr)
```

👉 The condition `element > 42` keeps **only 43 and 44**.

---

### Keep only even numbers

```
1. arr = np.array([1, 2, 3, 4, 5, 6, 7])
2. # Generate filter list based on condition
3. filter_arr = []
4. for element in arr:
5. if element % 2 == 0: # Check if even
6. a. filter_arr.append(True)
7. else:
8. a. filter_arr.append(False)
9. newarr = arr[filter_arr]
10. print(newarr)
```

👉 The condition `element % 2 == 0` keeps **only even numbers**.

---

## 3. Using NumPy's Built-in Feature

- Instead of using loops, **NumPy allows direct filtering**.

### Keep values greater than 42

```
1. arr = np.array([41, 42, 43, 44])
2. filter_arr = arr > 42 # This creates a Boolean array
3. newarr = arr[filter_arr]
```

```
4. print(newarr)
```

---

### Keep only even numbers

1. `arr = np.array([1, 2, 3, 4, 5, 6, 7])`
2. `filter_arr = arr % 2 == 0 # True for even numbers`
3. `newarr = arr[filter_arr]`
4. `print(newarr)`

👉 **NumPy automatically creates the Boolean list** without a loop.

---

### 💡 Summary

| Method              | Example                               | Description                           |
|---------------------|---------------------------------------|---------------------------------------|
| Boolean List        | <code>arr[[True, False, True]]</code> | Manually selects elements             |
| Loop with Condition | <code>arr[filter_arr]</code>          | Generates True/False list dynamically |
| NumPy Condition     | <code>arr[arr &gt; 42]</code>         | Direct filtering (Best way!)          |

---

### ✅ Best Practice

- Using **NumPy conditions directly** (`arr[arr > 42]`) is **faster** and **more efficient** than using loops.

### Find Min and Max values

- In **NumPy**, you can use the `min()` and `max()` functions to find the **minimum** and **maximum** values in an array.
- 

### Example: Finding Min & Max in a 1D Array

1. `import numpy as np`
  2. `arr = np.array([10, 20, 5, 8, 30])`
  3. `print("Minimum value:", np.min(arr)) # Output: 5`
  4. `print("Maximum value:", np.max(arr)) # Output: 30`
-

## Finding Min & Max in a 2D Array

1. `import numpy as np`
  2. `arr = np.array([[3, 7, 2],[5, 8, 1]])`
  3. `print("Minimum value:", np.min(arr))` # Output: 1
  4. `print("Maximum value:", np.max(arr))` # Output: 8
- 

## Finding Min & Max Along an Axis

### *Find min/max row-wise (axis=1)*

1. `arr = np.array([[3, 7, 2],[5, 8, 1]])`
2. `print("Min row-wise:", np.min(arr, axis=1))` # Output: [2 1]
3. `print("Max row-wise:", np.max(arr, axis=1))` # Output: [7 8]

### *Find min/max column-wise (axis=0)*

1. `print("Min column-wise:", np.min(arr, axis=0))` # Output: [3 7 1]
  2. `print("Max column-wise:", np.max(arr, axis=0))` # Output: [5 8 2]
- 

## Finding Index of Min & Max

- If you also need the **index** of the min/max value, use `argmin()` and `argmax()`.
1. `arr = np.array([10, 20, 5, 8, 30])`
  2. `print("Index of min value:", np.argmin(arr))` # Output: 2 (index of 5)
  3. `print("Index of max value:", np.argmax(arr))` # Output: 4 (index of 30)
- 

## Summary

- `np.min(arr)`, `np.max(arr)` → Min & max in an array
- `np.min(arr, axis=0)`, `np.max(arr, axis=1)` → Min & max along rows/columns
- `np.argmin(arr)`, `np.argmax(arr)` → Index of min & max values

## NumPy Universal Functions (ufuncs)-

- In NumPy, **ufuncs (universal functions)** are special functions that perform operations on entire arrays at once, making them much **faster** than regular Python loops.
- A ufunc takes one or more inputs, applies a mathematical operation, and returns an output **element-wise**
- For example, instead of using a for loop to add numbers in a list, NumPy can do it in **one step** using a ufunc

## 1.Introduction to ufuncs

- A ufunc takes one or more inputs, applies a mathematical operation, and returns an output **element-wise**.

### Example: Square root of numbers using ufunc

1. `import numpy as np`
2. `arr = np.array([4, 9, 16, 25])` # Creating an array
3. `result = np.sqrt(arr)` # Finding square root of each element
4. `print(result)` # Output: [2. 3. 4. 5.]

◆ Here, `np.sqrt()` is a ufunc that quickly calculates the square root of each number in the array.

## 2. Creating Custom ufuncs

- Can create your own ufuncs using `np.frompyfunc()`.

### Example: Creating a custom addition function

1. `def my_add(x, y):`
2. `return x + y`
3. `add_ufunc = np.frompyfunc(my_add, 2, 1)` # Takes 2 inputs, returns 1 output
4. `print(add_ufunc([1, 2, 3], [4, 5, 6]))` # Output: [5 7 9]

This allows us to define **custom** operations and apply them efficiently to arrays.

---

## 3. Basic Arithmetic with ufuncs

- NumPy has built-in ufuncs for basic math operations like **addition, subtraction, multiplication, and division**.

### Example: Performing arithmetic operations

1. `a = np.array([10, 20, 30])`
2. `b = np.array([1, 2, 3])`
3. `print(np.add(a, b))` # [11 22 33]
4. `print(np.multiply(a, b))` # [10 40 90]
5. `print(np.divide(a, b))` # [10. 10. 10.]

- No need for loops—NumPy does all calculations in one step!

---

## 4. Rounding Numbers

NumPy can **round numbers** in different ways:

- `np.around()` → Rounds to the nearest decimal place
- `np.floor()` → Rounds **down**
- `np.ceil()` → Rounds **up**

### Example: Rounding values

1. `arr = np.array([1.56, 2.34, 3.89])`
  2. `print(np.around(arr, decimals=1))` # [1.6 2.3 3.9]
  3. `print(np.floor(arr))` # [1. 2. 3.]
  4. `print(np.ceil(arr))` # [2. 3. 4.]
- 

## 5. Logarithms (Log)

Logarithms are used in calculations like **finding growth rates**.

- `np.log()` → Natural log
- `np.log2()` → Log base 2
- `np.log10()` → Log base 10

### Example: Logarithm calculations

1. `arr = np.array([1, 2, 10, 100])`
  2. `print(np.log10(arr))` # Log base 10
- Useful in **data science** and **mathematics**.
- 

## 6. Summations (Adding Numbers)

You can add all values in an array using:

- `np.sum()` → Adds all elements
- `np.cumsum()` → Cumulative sum

### Example: Finding the sum of an array

1. `arr = np.array([1, 2, 3, 4])`
2. `print(np.sum(arr))` # 10
3. `print(np.cumsum(arr))` # [1 3 6 10]

- 4. `cumsum()` keeps adding previous values.
- 

## 7. Products (Multiplication)

- `np.prod()` → Multiplies all values
- `np.cumprod()` → Cumulative product

### Example: Multiplying elements in an array

1. `arr = np.array([1, 2, 3, 4])`
  2. `print(np.prod(arr))` # 24
  3. `print(np.cumprod(arr))` # [1 2 6 24]
- `cumprod()` keeps multiplying previous values.
- 

## 8. Differences between Elements

- `np.diff()` finds **the difference between each pair of elements**.

### Example: Finding differences

1. `arr = np.array([10, 20, 30, 50])`
  2. `print(np.diff(arr))` # [10 10 20]
- Useful for **rate of change** calculations.
- 

## 9. Least Common Multiple (LCM)

- `np.lcm.reduce()` → Finds the **LCM** of an array.

### Example: Finding LCM

1. `arr = np.array([4, 5, 6])`
  2. `print(np.lcm.reduce(arr))` # 60
- LCM is the **smallest number divisible by all elements**.
-

## 10. Greatest Common Divisor (GCD)

- `np.gcd.reduce()` → Finds the **GCD** of an array.

### Example: Finding GCD

1. `arr = np.array([8, 12, 16])`
2. `print(np.gcd.reduce(arr))` # 4

- GCD is the **largest number that divides all elements**.
- 

## 11. Trigonometry (Sin, Cos, Tan)

NumPy has trigonometric functions like:

- `np.sin()`, `np.cos()`, `np.tan()`
- Convert degrees to radians using `np.deg2rad()`

### Example: Finding sine values

1. `angles = np.array([0, 30, 45, 60, 90])`
2. `radians = np.deg2rad(angles)`
3. `print(np.sin(radians))` # Sine values

- Important for **geometry and physics**.
- 

## 12. Hyperbolic Functions

These functions are useful in **science and engineering**:

- `np.sinh()`, `np.cosh()`, `np.tanh()`

### Example: Hyperbolic functions

1. `arr = np.array([0, 1, 2])`
  2. `print(np.sinh(arr))` # Hyperbolic sine
- 

## 13. Set Operations (Unique, Union, Intersection)

NumPy allows **set operations** on arrays:

- `np.union1d()` → Combines elements
- `np.intersect1d()` → Finds common elements
- `np.setdiff1d()` → Finds elements **only in the first set**

### Example: Set operations

1. `a = np.array([1, 2, 3, 4])`
2. `b = np.array([3, 4, 5, 6])`
3. `print(np.union1d(a, b))` # [1 2 3 4 5 6]
4. `print(np.intersect1d(a, b))` # [3 4]
5. `print(np.setdiff1d(a, b))` # [1 2]

## NumPy Random and Probability Distributions

### 1. NumPy Random

NumPy provides a module called `numpy.random` that is used to generate random numbers, perform random sampling, and manipulate probability distributions.

#### Key Features:

- Random number generation
- Permutations and shuffling
- Sampling from different distributions

### 2. Random Intro

The `numpy.random` module allows generating pseudo-random numbers for various use cases. These numbers are deterministic and can be reproduced using a seed.

#### Example:

1. `import numpy as np`
2. `np.random.seed(42)` # Set seed for reproducibility
3. `print(np.random.rand(3))` # Generate 3 random numbers between 0 and 1

### 3. Data Distribution

Data distribution refers to how data points are spread across a range of values. NumPy provides functions to sample from various probability distributions.

### 4. Random Permutation

Random permutation is used to shuffle arrays or return a randomly permuted sequence.

**Example:**

1. `arr = np.array([1, 2, 3, 4, 5])`
2. `np.random.shuffle(arr)` # Shuffles the array in-place
3. `print(arr)`

**Generating a permuted sequence:**

```
print(np.random.permutation(10))
```

## 5. Seaborn Module

Seaborn is a visualization library that integrates well with NumPy and pandas for statistical data visualization.

**Example:**

1. `import seaborn as sns`
2. `import matplotlib.pyplot as plt`
3. `sns.histplot(np.random.normal(size=1000), kde=True)`
4. `plt.show()`

## 6. Normal Distribution

A normal (Gaussian) distribution is a symmetric bell-shaped distribution defined by a mean ( $\mu$ ) and standard deviation ( $\sigma$ ).

**Example:**

```
np.random.normal(loc=0, scale=1, size=10) # Mean=0, Std=1
```

## 7. Binomial Distribution

A binomial distribution describes the number of successes in a fixed number of trials with a given probability of success.

**Example:**

```
np.random.binomial(n=10, p=0.5, size=10) # 10 trials, 50% success probability
```

## 8. Poisson Distribution

Poisson distribution models the number of events occurring in a fixed interval of time/space.

**Example:**

```
np.random.poisson(lam=3, size=10) # Average rate of occurrence = 3
```

## 9. Uniform Distribution

In a uniform distribution, every value has an equal probability of occurring.

### Example:

```
np.random.uniform(low=0, high=10, size=5)
```

## 10. Logistic Distribution

Logistic distribution is similar to normal but has heavier tails, making it useful for modeling growth.

### Example:

```
np.random.logistic(loc=0, scale=1, size=10)
```

## 11. Multinomial Distribution

Generalization of the binomial distribution for multiple categories.

### Example:

```
np.random.multinomial(n=10, pvals=[0.2, 0.3, 0.5], size=5)
```

## 12. Exponential Distribution

Models the time between events in a Poisson process.

### Example:

```
np.random.exponential(scale=1, size=10)
```

## 13. Chi Square Distribution

Used in statistical tests to determine variance among distributions.

### Example:

```
np.random.chisquare(df=2, size=10)
```

## 14. Rayleigh Distribution

Used in signal processing and physics applications.

### Example:

```
np.random.rayleigh(scale=2, size=10)
```

## 15. Pareto Distribution

Models wealth distribution and other real-world phenomena.

### **Example:**

```
np.random.pareto(a=3, size=10)
```

## 16. Zipf Distribution

Models frequencies of words, web clicks, etc.

### **Example:**

```
np.random.zipf(a=2, size=10)
```